

Nagare Smaller/Faster Evidence and Architecture

HyMeKo / Nagare Working Report

Created-at: 2026-07-01 16:15 JST

Executive Summary

Nagare is already much smaller than PyTorch for the small deployed CPU workloads tested here. It is also faster when the computation is expressed as native, allocation-light kernels. The strongest positive case is the direct Chebyshev deploy classifier with Chebyshev-domain rescale only, where Nagare matches PyTorch logits within approximately 10^{-7} , runs $1.68\times$ – $1.86\times$ faster, and uses roughly 5.39 MiB peak RSS versus a PyTorch process-tree sample of 552.89 MiB.

The important negative case is equally useful: the unfused global-pool entropy-feedback forward is smaller in Nagare but slower than PyTorch, because the current Rust path materializes the wide broadcast tensor $[h, \text{pool}(h), H]$. That points to a concrete next kernel: a fused entropy-feedback update that streams pooled context and entropy directly into the update linear.

1 Measured Results

1.1 Exact Synthetic Chebyshev Classifier

The model is a small classifier $2 \rightarrow 32 \rightarrow 2$ with direct Chebyshev deploy activation $k = 5$. The pre-curve adapter is Chebyshev-domain rescale only: $z \mapsto 0.5z$, with no tanh bound. PyTorch generated the data and weights for moons, spiral, and XOR. Nagare parsed the same text fixture and ran the same forward pass.

Task	Max abs logits	PyTorch median	Nagare median	Speedup
Moons	7.45×10^{-8}	4.05 us	2.18 us	$1.86\times$
Spiral	8.94×10^{-8}	5.43 us	3.10 us	$1.75\times$
XOR	1.04×10^{-7}	4.27 us	2.54 us	$1.68\times$

Table 1: Exact PyTorch/Nagare CPU forward comparison on identical synthetic fixtures.

Engine	Peak RSS
PyTorch process tree	552.89 MiB
Nagare release executable	5.39 MiB

Table 2: Resident memory sample for the synthetic Chebyshev classifier comparison.

1.2 Native CR and Chebyshev-CR Kernels

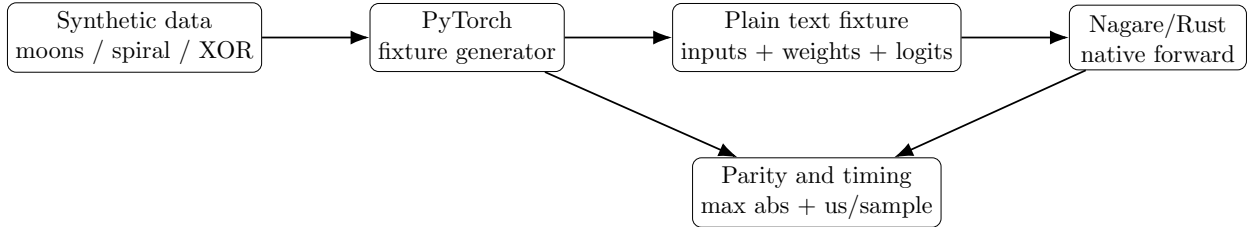
Native Catmull–Rom (CR), Chebyshev-CR train mode, and direct Chebyshev deploy were measured on small descriptor-like tensors.

Case	Shape	CR train	Cheb-CR train	Cheb deploy
Tiny descriptor	$n = 1, c = 16$	1.1000 us	2.1000 us	0.4000 us
Small descriptions	$n = 8, c = 32$	1.0750 us	0.8875 us	0.2375 us
Toy point batch	$n = 4608, c = 32$	1.0717 us	1.0013 us	0.4043 us

Table 3: Median CPU forward time per sample for native Nagare activations.

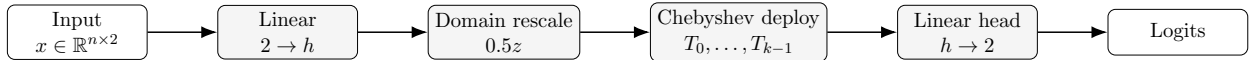
2 Architecture

2.1 Exact Fixture Comparison



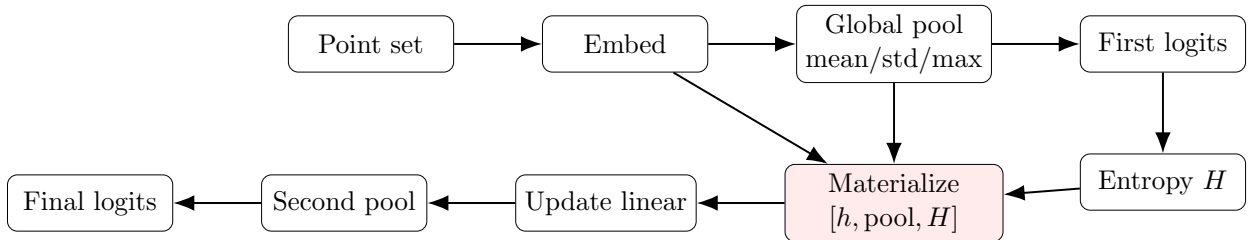
The fixture path makes the comparison strict: PyTorch and Nagare receive the same samples, the same weights, and the same expected logits. Accuracy here is only a sanity check because the weights are random; the real metric is exact forward parity and runtime.

2.2 Native Chebyshev Deploy Classifier



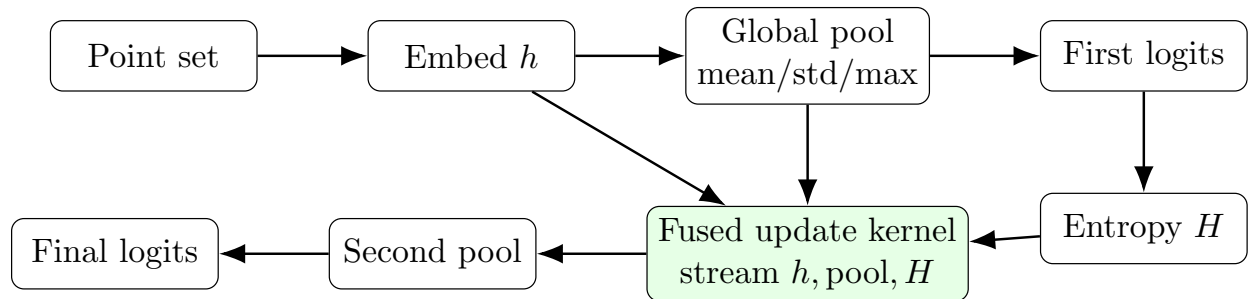
This is the architecture where Nagare is both smaller and faster. The direct Chebyshev deploy kernel avoids CR gather, and the pre-curve adapter is a scale into the Chebyshev domain rather than a saturating neural activation. After removing a per-scalar temporary allocation, deploy runs in the sub-microsecond to low-microsecond range on the measured small shapes.

2.3 Current Entropy-Feedback Path



This is the architecture where Nagare is smaller but currently slower. The red node is the problem: it allocates a wide broadcast tensor and makes the native Rust path resemble a PyTorch-style composed tensor program.

2.4 Target Fused Entropy-Feedback Path



The target kernel should compute the update linear directly from the local embedding row, pooled context, and entropy scalar. It should not allocate $[h, \text{pool}, H]$. This is the next step needed to make the entropy-feedback architecture match the smaller-and-faster behavior observed for Chebyshev deploy.

3 Decision

The result should be used as a development gate:

1. Promote direct Chebyshev deploy as the small CPU inference path.
2. Keep CR and Chebyshev-CR train mode as native training-quality operators.
3. Implement fused entropy-feedback update before making any broader speed claim for the global-pool entropy model.
4. Re-run exact PyTorch parity after the fused update kernel lands.

4 Evidence Trail

- `reports/2026-07-01-nagare-pytorch-synthetic-cheby-compare.md`
- `reports/2026-07-01-nagare-cr-cheby-small-perf.md`
- `reports/2026-07-01-nagare-pytorch-global-pool-entropy-parity.md`
- `reports/2026-07-01-nagare-smaller-faster-summary.md`